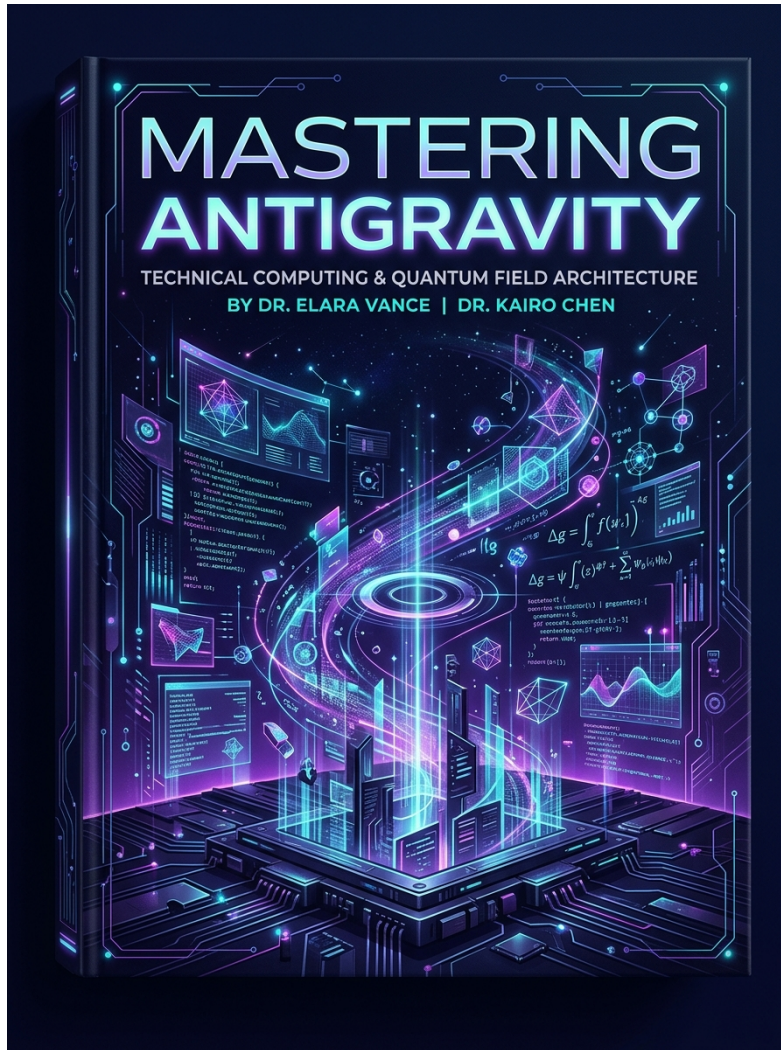


MASTERING ANTIGRAVITY

The Definitive Guide to AI-First Software Engineering

Expanded Edition: Prompts, Goal Architecture & The Modern AI Developer Toolkit



Sudheer K. Mohammed

Comprehensive Reference Edition • 2026

Foreword: Escaping the Gravity Well

For over seven decades, software development has been bound by an inescapable law of cognitive gravity. Every line of code written demanded an equal measure of mental overhead: keeping track of transitive dependency conflicts, syntax quirks, mutating framework lifecycles, and arcane compiler flags. As codebases ballooned from tens of thousands of lines into multimillion-line monorepos, developer productivity became chained to the floor.

When LLMs emerged, early attempts simply dropped autocomplete boxes into old text editors. They were pleasant novelties, but they did not solve the fundamental crisis: software engineering is not typing; it is **reasoning, planning, executing, and verifying**.

Google Antigravity (AGY) represents the escape velocity. Antigravity treats artificial intelligence not as an assistant that guesses your next twenty characters, but as a **peer engineer** equipped with a sandboxed terminal, code navigation ASTs, headless browser automation, and strict safety governance.

This expanded edition is designed as the ultimate engineering manual. It does not simply explain theories: it breaks down the exact architectural differences between the **CLI**, the **IDE**, the **2.0 Desktop Canvas**, and the **Python SDK**, presents battle-tested prompt blueprints, establishes the formal mathematics of goal definition, and charts the essential skill stack required for software engineers to thrive in the agentic era.

Contents

Foreword: Escaping the Gravity Well	i
I The Ground Floor (Beginner)	1
1 The AI-First Awakening	2
1.1 Beyond the Digital Typewriter	2
1.2 The Architectural Triad of Antigravity	2
1.3 The Four Operational Surfaces of Antigravity	3
2 First Contact: Setup, CLI & Configuration	4
2.1 System Prerequisites & Installation	4
2.2 Authentication and Workspaces	4
2.3 Mastering CLI Flags and Headless Scripting	4
3 The Three Faces of Antigravity	6
3.1 The Antigravity IDE: Three AI Modalities	6
3.1.1 1. Passive: Antigravity Tab (Autocomplete & Supercomplete)	6
3.1.2 2. Instructive: Inline Command (Ctrl+I / Cmd+I)	6
3.1.3 3. Collaborative: Sidebar Chat & Full Agent Mode	6
3.2 Antigravity 2.0: The Desktop Command Center	7
3.3 Deep Dive: CLI vs IDE vs SDK in Real Engineering	7
II Core Agentic Workflows & Pair Programming (Intermediate)	8
4 The Art of Planning, Goal Architecture & Review Loops	9
4.1 Why Unplanned Agents Fail	9
4.2 The Anatomy of Planning Mode	9
4.3 The Anatomy of an Unbreakable Goal Definition	10
4.3.1 Vague Goal vs. Production-Grade ATDG Goal	10
4.4 The /grill-me Protocol	10
5 Precision Context Engineering & Battle-Tested Prompts	12
5.1 The Context Economy	12
5.2 The Power of @ Mentions	12
5.3 The Production Prompt Engineering Taxonomy	12
5.3.1 1. The Zero-Regression Refactoring Blueprint	12
5.3.2 2. The Forensic Bug Triage Blueprint	13
5.3.3 3. Negative Constraints Formulation	13

6	The Iron Sandbox: Security & Permissions	15
6.1	Execution Policies	15
6.2	Project-Level Allow and Deny Lists	15
III	The Customization Engine (Advanced)	17
7	Rules of the Realm: Hierarchical Governance	18
7.1	The Role of Rules	18
7.2	Hierarchical Directory Walking	18
7.3	Trigger Modes: <code>always_on</code> vs <code>model_decision</code>	18
8	Crafting Skills with Progressive Disclosure	20
8.1	What is a Skill?	20
8.2	The Architecture of Progressive Disclosure	20
8.3	Complete Skill Implementation	20
9	Deterministic Gates with Hooks	22
9.1	Configuring <code>hooks.json</code>	22
9.2	Writing a Blocking Pre-Tool Validation Hook	22
10	Model Context Protocol (MCP) Integration	24
10.1	What is MCP?	24
10.2	Configuring <code>mcp_config.json</code>	24
10.3	Lazy vs. Eager Tool Loading	25
IV	Programmatic Agent Mastery & The Skill Stack (Expert)	26
11	The Antigravity Python SDK	27
11.1	Headless Agent Orchestration	27
11.2	Building an Asynchronous SRE Agent	27
11.3	Deep Architectural Comparison: CLI vs. SDK	29
12	Swarms, Browser Agents & Daemons	30
12.1	Subagent Delegation (<code>invoke_subagent</code>)	30
12.2	Autonomous Browser Automation	30
12.3	Background Daemons & The Scheduler	30
13	Enterprise Monorepos & CI/CD	32
13.1	Scaling Antigravity in Large Repositories	32
13.2	GitHub Actions CI/CD Integration	32
14	The AI Developer's Skill Stack: Thriving in the Agentic Era	34
14.1	The Evolution: From Code Typist to Systems Director	34
14.2	The Six Core Pillars of AI Engineering	34
14.2.1	1. Solving the Oracle Problem	35
14.2.2	2. The Tri-Layer Governance Stratum	35
15	Interfacing Antigravity: WhatsApp, Telegram & Google Workspace	36
15.1	The Omnichannel Agent Architecture	36
15.2	WhatsApp Cloud API Integration Workflow	36
15.2.1	Production WhatsApp Gateway Server	36

15.3 Telegram Bot Integration Workflow	38
15.3.1 Real-Time Thinking Stream over Telegram	38
15.4 Google Workspace Integration (Sheets & Gmail via MCP)	39
15.4.1 1. Google Sheets as an Autonomous Audit Ledger	39
15.4.2 2. Automated Bug Ledger & Executive Email Workflow	40
Appendix A: Slash Command Quick Reference	41
Appendix B: Customization Directory Reference	42
Appendix C: Keyboard Shortcuts Cheat Sheet	43
Appendix D: Production Prompt & Goal Cookbook	44

Part I

The Ground Floor (Beginner)

Chapter 1

The AI-First Awakening

1.1 Beyond the Digital Typewriter

To understand Antigravity, one must first unlearn the last twenty years of editor philosophy. The classical IDE—born of Smalltalk, refined by Eclipse, and popularized by Visual Studio Code—is built on a singular premise: *a human sits at a keyboard, manually edits buffer strings, and occasionally asks external programs (compilers, linters) to evaluate those strings.*

When AI was first introduced to this paradigm, it was relegated to the status of an advanced spellchecker. Copilot-style tools operated under strict token-completion constraints: given the cursor location L in document D , predict tokens $t_1, t_2, \dots, t_n$.

This passive modality suffers from three fatal flaws:

1. **Zero Architectural Intention:** The autocomplete model does not know *why* you are writing the loop, only that the loop commonly follows the preceding assignment.
2. **Blindness to the Build Loop:** When an autocomplete tool inserts an import statement that does not exist in `package.json`, it cannot check the error, run `npm install`, or read the resulting error log.
3. **Cognitive Whiplash:** The human engineer spends more time reading and mentally debugging subtle AI hallucination snippets than they would have spent writing the code from scratch.

1.2 The Architectural Triad of Antigravity

Antigravity re-architects the software development environment into three interconnected components:

Core Architecture Concept: The Cognitive Engine

Unlike standard conversational agents that respond in single back-and-forth turns, Antigravity executes an internal **Plan-Act-Observe Loop**. The model analyzes your codebase, formulates a multi-step execution tree, invokes filesystem and shell tools, observes the stdout/stderr return values, and self-corrects until its objective is verified.

1. **The Core Reasoning Runtime:** Backed by multimodal Gemini models with multi-million token context windows. It processes code, terminal logs, UI screenshots, and git trees simultaneously.



Figure 1.1: The Antigravity Agent Architecture: Decoupling reasoning from user interfaces, integrating live terminal sandboxes, compiler feedback loops, and intelligent context routing.

- 2. **The Execution Sandbox:** An isolated process environment where the agent can run shell commands (`make`, `cargo test`, `docker build`) without endangering your host OS or accessing unapproved networks.
- 3. **The Customization Engine:** A tiered system composed of **Rules**, **Skills**, **Hooks**, and **MCP Servers** that allow any repository to instill its own engineering culture into the agent.

1.3 The Four Operational Surfaces of Antigravity

Antigravity delivers its intelligence across four distinct form-factors:

Table 1.1: Comparative Matrix: The Four Antigravity Surfaces

Dimension	CLI (agy)	Antigravity IDE	Antigravity 2.0	Python SDK
Form Factor	Terminal TUI	VS Code Fork	Electron Desktop	Python Library
Ideal User	DevOps, SREs, CLI ninjas	Full-stack Developers	Tech Leads, Architects	Automation Engineers
Modality	Turn-based	Tab, Inline, Chat	Canvas & Artifacts	Headless Event Streams
Host Footprint	Minimal (< 50MB)	Moderate (350MB)	Moderate (400MB)	Embedded in Python
Interactive?	Yes (REPL)	Yes (In-editor)	Yes (Multi-pane)	Programmatic (Async)
Scriptable?	Yes (Flags)	No (GUI focused)	No (GUI focused)	Completely (Pipelines)

Chapter 2

First Contact: Setup, CLI & Configuration

2.1 System Prerequisites & Installation

Antigravity runs natively on modern 64-bit platforms: Linux (x86_64 / ARM64), macOS (Apple Silicon / Intel), and Windows (via WSL2).

To install the Antigravity Command Line Interface (**agy**):

```
1 # Verify system environment
2 uname -m
3
4 # Install Antigravity CLI via standard package channel
5 curl -fsSL https://antigravity.google/install.sh | bash
6
7 # Verify installation
8 agy --version
```

Listing 2.1: Installing and verifying the Antigravity CLI

2.2 Authentication and Workspaces

On your first run of **agy**, the runtime initiates an OAuth device-authorization flow. A secure browser tab opens, allowing you to link your Google Cloud or Vertex AI developer identity.

Once authenticated, your runtime secrets and tokens are encrypted under:

- Linux/macOS: `~/.gemini/antigravity-cli/`
- Global Configuration: `~/.gemini/config/`

```
1 # Launch interactive REPL in current directory
2 cd ~/projects/my-microservice
3 agy
4
5 # Or launch directly with an initial instruction
6 agy "Review open git diff and run unit tests"
```

Listing 2.2: Launching your first agent session

2.3 Mastering CLI Flags and Headless Scripting

The CLI provides powerful flags for scriptability and continuous integration:

- `agy -w /path/to/project`: Explicitly specifies the working workspace root.
- `agy -model gemini-3.8-pro`: Overrides the active reasoning model.
- `agy -sandbox=strict`: Forces all bash tools to run inside an ephemeral sandbox container.
- `agy -approval=never`: Runs the agent fully autonomously (ideal for headless CI/CD runners).
- `agy -format=json`: Emits machine-parseable JSON streams of tool calls and output events.

Engineering Lab Project: Project 1: WeatherCraft - CLI Bootstrapping

Let us walk through using `agy` in the terminal to initialize a full-stack project from scratch without writing boilerplate manually:

```
1 # In your terminal
2 mkdir weathercraft && cd weathercraft
3 agy "Initialize a FastAPI backend and a Next.js 14 frontend in
    ./backend and ./frontend. Configure CORS between
    localhost:3000 and 8000, write a health check endpoint, and
    verify it with pytest."
```

The agent will create the virtual environment, write `main.py`, set up `requirements.txt`, run `pip install`, generate `test_main.py`, and execute `pytest` inside the terminal sandbox to prove it works before returning control to you.

Chapter 3

The Three Faces of Antigravity

Antigravity meets engineers across three complementary surfaces: the in-editor IDE, the standalone 2.0 Desktop canvas, and the terminal TUI.

3.1 The Antigravity IDE: Three AI Modalities

Built on top of the open VS Code core, the Antigravity IDE integrates agentic capabilities directly into your editing canvas across three distinct interaction levels:

3.1.1 1. Passive: Antigravity Tab (Autocomplete & Supercomplete)

Operating at the single-keystroke layer, Tab Autocomplete predicts your next intent:

- **Contextual Autocomplete:** Suggests tokens at the cursor based on adjacent open tabs, diagnostics, and recent terminal outputs.
- **Supercomplete:** Generates multi-line diffs (including deletions and structural refactors) rendered in floating preview overlays.
- **Tab-to-Jump:** Predicts where your cursor needs to navigate next (e.g., jumping from a function definition to its unit test assertion) and teleports your cursor with a single `<Tab>`.
- **Tab-to-Import:** Automatically resolves missing symbols and injects import statements at the top of the file.

3.1.2 2. Instructive: Inline Command (`Ctrl+I` / `Cmd+I`)

When you want targeted changes without switching focus to a chat window:

1. Highlight a block of code (e.g., a messy SQL query or an unoptimized regex).
2. Press `Ctrl+I` (or `Cmd+I` on macOS).
3. Type: *"Refactor this into parameterized sqlx query with error logging"*.
4. Review the localized red/green diff right in your editor buffer and press `Enter` to accept.

3.1.3 3. Collaborative: Sidebar Chat & Full Agent Mode

When a task spans across files, requires terminal execution, or needs architectural deliberation, press `Ctrl+Shift+L` to engage the Sidebar Agent. The agent can read files, write new modules, execute `npm test`, diagnose failures, and consult online documentation.

3.2 Antigravity 2.0: The Desktop Command Center

Antigravity 2.0 is a standalone Electron desktop application tailored for high-autonomy multi-agent orchestration. It decouples agent execution from your text editor:

- **Left-Hand Sidebar:** Manage multiple project workspaces, active background cron schedules, installed skills, and global security policies.
- **Chat Canvas:** Supports deep markdown artifacts, interactive review modals, and media uploads.
- **Auxiliary Telemetry Pane:** A multi-tab inspector showing live Subagent swarms, running background daemon logs, generated Artifact documents, and attached terminal sessions.

3.3 Deep Dive: CLI vs IDE vs SDK in Real Engineering

Pro Tip: When to use which surface?

Use the **IDE** when you are actively writing code, navigating files, and need inline diffs and autocomplete. Use the **CLI (agy)** when you are logged into remote cloud VMs, managing servers over SSH, or chaining shell pipes. Use the **Desktop 2.0 Canvas** when conducting architectural redesigns, planning complex initiatives, or monitoring multiple background subagents. Use the **Python SDK** when embedding agent intelligence into CI/CD pipelines, Slack bots, or automated test runners.

Part II

Core Agentic Workflows & Pair Programming (Intermediate)

Chapter 4

The Art of Planning, Goal Architecture & Review Loops

4.1 Why Unplanned Agents Fail

The most common rookie mistake with AI coding assistants is issuing massive, ambiguous prompts:

"Rewrite my monolithic express app to clean architecture with TypeScript and add Docker."

An unplanned agent immediately begins hacking files. By step 14, it has broken TypeScript compilation. By step 25, it is deleting code to silence compiler errors. By step 40, your git working tree is ruined.

Antigravity solves this with formal **Planning Mode**.

Pro Tip: The Planning Golden Rule

Never let an agent touch code on a multi-file task until it has produced a formal `implementation_plan.md` artifact and received your explicit sign-off.

4.2 The Anatomy of Planning Mode

When confronted with architectural tasks, Antigravity enters Planning Mode:

1. **Discovery & Research:** The agent searches the project using read-only tools (`grep_search`, `view_file`, `list_dir`). It inspects configuration files, existing test conventions, and database schemas.
2. **Implementation Plan Artifact:** The agent drafts a comprehensive technical plan at `brain/<id>/implementation_plan.md`.
3. **User Review Gate:** The agent halts execution and presents an interactive review card. You can request changes or approve the plan with a click.
4. **Execution & Verification:** Once approved, the agent executes edits incrementally, running build commands at each stage.
5. **Walkthrough Artifact:** Upon completion, the agent creates `walkthrough.md`, documenting every change made and providing proof of verification tests.

4.3 The Anatomy of an Unbreakable Goal Definition

When running agents autonomously using the `/goal` command, the primary cause of runaway compute or degradation is **unbounded goal ambiguity**. If an agent does not have a mathematically deterministic stopping condition, it will hallucinate micro-optimizations indefinitely.

To prevent this, production Antigravity engineers use the **ATDG Framework (Acceptance Test-Driven Goal)**:

Core Architecture Concept: The Four Pillars of an ATDG Goal

Every autonomous goal statement must define:

1. **The Invariant:** What existing behavior or contract must NOT change under any circumstance.
2. **The Blast Radius:** The exact subdirectories or files that are mutable (everything else is read-only).
3. **The Deterministic Oracle:** The exact terminal command that evaluates success (must return exit code 0).
4. **The Stopping Rule:** The explicit condition under which the agent must stop and declare victory.

4.3.1 Vague Goal vs. Production-Grade ATDG Goal

Compare the two following goal formulations:

Production Prompt Template: Vague Goal (Fails 80% of the time)

```
/goal Fix the broken checkout tests and make sure the API is robust.
```

Production Prompt Template: ATDG Production Goal (Succeeds 99% of the time)

```
/goal Invariant: Do NOT modify existing database schemas in
prisma/schema.prisma or change the public response payload of POST
/api/checkout.
Scope: You are only permitted to edit files under src/services/checkout/ and
src/utils/pricing.ts.
Oracle: Run 'npm test test/checkout.spec.ts'.
Termination: When all 14 tests pass with exit code 0 and 'npm run lint' yields zero
errors, stop and generate walkthrough.md.
```

4.4 The /grill-me Protocol

If you know *what* feature you want but haven't settled on the edge cases, invoke the `/grill-me` slash command.

Instead of jumping into code, Antigravity assumes the role of a meticulous Principal Architect:

- *"How should we handle database connection timeouts during token refresh?"*

- *"Should we invalidate all active user sessions upon password reset, or only the current device?"*
- *"Do you want this API endpoint to be rate-limited by IP or by authenticated user ID?"*

Only once all design ambiguities are resolved does it formulate the plan.

Engineering Lab Project: Project 2: AuthSentinel - Refactoring Legacy Auth in Planning Mode

In this project, we refactor an insecure session-cookie service into an audited JWT architecture with rotating refresh tokens stored in Redis:

1. Open the project in Antigravity IDE and type `/grill-me` in Sidebar Chat: *"Refactor auth from memory sessions to Redis JWT with refresh token rotation."*
2. The agent interviews you on secret rotation windows and cookie security flags (`HttpOnly`, `SameSite=Strict`).
3. The agent generates `implementation_plan.md` outlining the exact modifications across `auth.service.ts`, `redis.client.ts`, and `auth.guard.ts`.
4. You review and approve the plan. The agent updates dependencies via `npm install`, writes the files, and launches `npm test` to verify that all 18 security regression tests pass.

Chapter 5

Precision Context Engineering & Battle-Tested Prompts

5.1 The Context Economy

Modern LLMs boast massive context windows, but context is not free:

- 1. **Latency:** Processing a 500,000-token prompt takes noticeably longer than a targeted 10,000-token prompt.
- 2. **Attention Dilution:** When irrelevant files clutter the context window, the model’s ability to notice critical subtleties in core logic degrades.

5.2 The Power of @ Mentions

Antigravity gives you surgical control over context using the @ mention system:

Table 5.1: Context Mention Resolvers in Antigravity

Mention Syntax	Resolved Context Injected
@file:src/auth.ts	Injects the full text of <code>src/auth.ts</code> into the turn.
@folder:src/controllers/	Injects directory listing and file summaries for that folder.
@symbol:UserService	Locates and injects the AST definition of the class or interface.
@commit:HEAD~1	Injects the git diff and commit log of the specified revision.
@transcript:0e58b2	Injects key context from an earlier conversation session.
@mcp:github/search_issues	Injects schema and tool instructions for an MCP tool.

5.3 The Production Prompt Engineering Taxonomy

Prompting an autonomous agent is fundamentally different from prompting a conversational chatbot. An agent executes real tools. Below are four battle-tested production prompt patterns:

5.3.1 1. The Zero-Regression Refactoring Blueprint

When refactoring critical business logic, you must forbid interface breakage and speculative deletions:

Production Prompt Template: Zero-Regression Refactoring

You are refactoring `@file:src/services/billing.py` to use `async/await` with `httpx` instead of `requests`.

Strict Invariants:

1. Preserve all public method signatures, argument names, and return type hints.
2. Preserve all existing docstrings and inline business comments.
3. Do NOT delete existing helper methods; mark them `@deprecated` if replaced.
4. Run `'pytest tests/test_billing.py'` before and after edits. All 22 tests must remain green.

5.3.2 2. The Forensic Bug Triage Blueprint

When diagnosing production errors, force the agent to reproduce the bug with a test *before* writing a fix:

Production Prompt Template: Forensic Triage Protocol

Investigate error log `@file:logs/sentry_event_9402.json`.

Follow this mandatory 3-step triage loop:

1. **Step 1 (Reproduction):** Write a reproduction unit test in `tests/repro_bug_9402.py` that asserts the expected behavior and currently FAILS. Run `pytest` to demonstrate the failure.
2. **Step 2 (Root-Cause Isolation):** Inspect `@symbol:calculate_rebate` and explain the exact mathematical flaw.
3. **Step 3 (Remediation):** Implement the minimal fix in `src/rebate.py`. Verify that `tests/repro_bug_9402.py` now passes and no existing tests regress.

5.3.3 3. Negative Constraints Formulation

Negative constraints prevent common LLM degradation behaviors:

- **Anti-Mocking:** *"Do NOT mock the database in integration tests; use the local testcontainers Postgres instance."*
- **Anti-Stubbing:** *"Never leave `// TODO: implement later stubs` or placeholder functions. Every branch must be fully implemented."*
- **Scope Jail:** *"Do NOT create files outside `src/features/search/`. Do NOT modify `package.json` without asking."*

War Story from the Trenches: The Million-Token Hallucination Trap

A development team once attached their entire 2GB `node_modules` folder to an agent prompt via a misplaced wildcard. The agent spent 12 minutes digesting 800,000 tokens of minified JavaScript, completely missed the two-line syntax bug in the user's controller, and timed out. By switching to `@symbol:UserController` and `@file:src/routes.ts`,

the context dropped to 1,400 tokens, and the bug was fixed in 4 seconds.

—

Chapter 6

The Iron Sandbox: Security & Permissions

Caution & Security Warning: The Threat Model of Autonomous Agents

An agent capable of executing terminal commands can accidentally run `rm -rf /`, leak environment secrets to public pastebins, or download malicious dependencies. Security must be enforced at the runtime level, not left to model obedience.

6.1 Execution Policies

Antigravity enforces configurable permission levels:

- **strict**: The agent must request interactive user approval before running any command, reading unapproved files, or touching the network.
- **request-review**: Read operations proceed automatically; write operations and shell commands require user approval.
- **proceed-in-sandbox**: The agent executes commands freely, but strictly inside an unprivileged Linux container sandbox with isolated namespaces.
- **always-proceed**: Zero confirmation prompts. Intended exclusively for isolated VM sandboxes or test runners.

6.2 Project-Level Allow and Deny Lists

Configure security controls in `.agents/settings.json`:

```
1 {
2   "security": {
3     "terminalSandbox": true,
4     "nonWorkspaceAccess": "deny",
5     "internetAccess": "allow",
6     "commandAllowlist": [
7       "git *",
8       "npm test*",
9       "cargo check",
10      "go test ./**"
11    ],
12    "commandDenylist": [
13      "rm -rf *",
14      "curl * | bash",
```

```
15         "sudo *",  
16         "dd *"  
17     ]  
18 }  
19 }
```

Listing 6.1: Hardening Antigravity security policies

Part III

The Customization Engine (Advanced)

Chapter 7

Rules of the Realm: Hierarchical Governance

7.1 The Role of Rules

While conversational prompts are ephemeral, **Rules** are permanent laws governing agent behavior. Rules enforce:

- Coding style guides (e.g., "Always use TypeScript strict mode; no **any**").
- Security mandates (e.g., "Never hardcode passwords or API secrets").
- Operational standards (e.g., "Every database query must use parameterized statements").

7.2 Hierarchical Directory Walking

Antigravity searches for rule files starting from the directory of the file being edited all the way up to the project root:

```
src/services/billing/payment.ts
↓
src/services/billing/AGENTS.md
↓
src/services/AGENTS.md
↓
src/AGENTS.md
↓
.agents/rules/*.md & GEMINI.md
```

All rules along this path are aggregated and applied, allowing micro-rules to govern specific packages without polluting global context.

7.3 Trigger Modes: `always_on` vs `model_decision`

Rules use YAML frontmatter to control their activation:

```
1 ---
2 trigger: always_on
3 ---
4 # Security Mandate
5 - Never commit plaintext secrets or API keys.
6 - Always use process.env for credentials.
```

```
7 - Reject any user inputs that bypass SQL injection validation.
```

Listing 7.1: Example Rule: .agents/rules/security.md

```
1 ---
2 trigger: model_decision
3 description: Best practices for React component rendering, useMemo,
   and useCallback.
4 ---
5 # React Performance Guidelines
6 - Do not wrap primitive values in useMemo.
7 - Always provide key attributes derived from stable IDs, never array
   indices.
```

Listing 7.2: Example Rule: .agents/rules/react-performance.md

Chapter 8

Crafting Skills with Progressive Disclosure

8.1 What is a Skill?

A **Skill** is a specialized procedure package. While rules state what *not* to do, skills teach the agent *how* to accomplish a complex, multi-step engineering procedure (such as deploying a Kubernetes cluster, executing a database migration, or profiling CPU bottlenecks).

8.2 The Architecture of Progressive Disclosure

Loading 40 skills with full markdown instructions into the initial system prompt would consume 100,000 tokens before the user even types a word.

Antigravity solves this with **Progressive Disclosure**:

1. **Registration Phase**: Only the **name** and **description** from the skill's YAML frontmatter are injected into the agent's initial tool registry.
2. **Activation Phase**: When a user's prompt matches the description, the agent executes its internal `view_file` tool to read the skill's `SKILL.md` file on-demand.

8.3 Complete Skill Implementation

Let us build a real-world production skill for automated database migrations:

```
1 ---
2 name: db-migrate
3 description: Manages PostgreSQL database migrations using Goose.
               Triggers on requests involving database migrations, schema
               changes, or running migrate up/down.
4 ---
5
6 # PostgreSQL Migration Workflow
7
8 ## Prerequisites
9 Verify database connectivity:
10 ```bash
11 goose postgres "$DATABASE_URL" status
12 ```
13
14 ## Step-by-Step Procedure
15 1. **Inspect Pending Migrations**:
16     Run status to check unapplied SQL scripts:
```

```
17  '''bash
18  goose -dir ./migrations postgres "$DATABASE_URL" status
19  '''
20  2. **Execute Dry-Run & Validation**:
21  Review the SQL statements in the pending migration file. Ensure
    every 'Up' statement has a corresponding 'Down' rollback
    statement.
22  3. **Apply Migration**:
23  '''bash
24  goose -dir ./migrations postgres "$DATABASE_URL" up
25  '''
26  4. **Verify Table Schema**:
27  Verify the schema changes using psql:
28  '''bash
29  psql "$DATABASE_URL" -c "\d+ users"
30  '''
```

Listing 8.1: File: .agents/skills/db-migrate/SKILL.md

Engineering Lab Project: Project 3: CloudScale - Authoring Enterprise Skills and MCP

In this project, we package an internal release automation skill that validates container images against Trivy vulnerability scanners, deploys to staging via Helm, runs integration smoke tests, and updates the Jira release ticket via MCP tools.

Deterministic Gates with Hooks

Core Architecture Concept: Probabilistic vs. Deterministic Guardrails

A prompt or rule is *probabilistic*: the LLM tries to follow it, but there is always a non-zero chance of a mistake. A **Hook** is *deterministic*: it is a shell script executed by the operating system before or after a tool call. If the hook fails, the action is stopped dead in its tracks.

9.1 Configuring `hooks.json`

Hooks are configured in `.agents/hooks.json`:

```
1 {
2   "hooks": {
3     "pre_tool_use": [
4       {
5         "matcher": { "tool": "run_command" },
6         "command": "python3 .agents/hooks/validate_command.py",
7         "blocking": true,
8         "timeoutMs": 5000
9       }
10    ],
11    "post_tool_use": [
12      {
13        "matcher": { "tool": "replace_file_content" },
14        "command": "npx prettier --write\n\"$ANTIGRAVITY_MODIFIED_FILE\"",
15        "blocking": false
16      }
17    ]
18  }
19 }
```

Listing 9.1: Configuring lifecycle hooks in `.agents/hooks.json`

9.2 Writing a Blocking Pre-Tool Validation Hook

Here is a Python script that prevents the agent from running destructive Git commands on protected branches:

```
1 import os
2 import sys
```

```
3 import json
4 import subprocess
5
6 # Read tool invocation details from environment
7 command_to_run = os.environ.get("ANTIGRAVITY_COMMAND_LINE", "")
8
9 # Check current git branch
10 branch = subprocess.check_output(
11     ["git", "rev-parse", "--abbrev-ref", "HEAD"],
12     text=True
13 ).strip()
14
15 if branch in ["main", "master", "production"]:
16     if "push" in command_to_run or "reset --hard" in command_to_run:
17         sys.stderr.write(
18             f"ERROR: Destructive command blocked on protected branch\n"
19             f'    {branch}'
20         )
21         sys.exit(1) # Non-zero exit code halts tool execution
22 sys.exit(0)
```

Listing 9.2: File: .agents/hooks/enforce_branch_protection.py

Chapter 10

Model Context Protocol (MCP) Integration

10.1 What is MCP?

The **Model Context Protocol (MCP)** is an open, standardized protocol created to connect AI agents with external data stores, SaaS APIs, and custom tooling over JSON-RPC (via standard I/O or SSE).

Through MCP, Antigravity can directly:

- Query production SQL databases.
- Inspect and review GitHub Pull Requests and Issues.
- Fetch CloudWatch or Datadog telemetry logs.
- Trigger AWS Lambda or Google Cloud Run deployments.

10.2 Configuring `mcp_config.json`

Add server definitions to `.agents/mcp_config.json`:

```
1 {
2   "mcpServers": {
3     "github": {
4       "command": "npx",
5       "args": ["-y", "@modelcontextprotocol/server-github"],
6       "env": {
7         "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_secureToken12345"
8       }
9     },
10    "production_postgres": {
11      "command": "docker",
12      "args": [
13        "run", "-i", "--rm",
14        "-e",
15        "DATABASE_URL=postgres://readonly@db.internal:5432/prod",
16        "mcp/postgres"
17      ]
18    }
19  }
```

Listing 10.1: Connecting external tools via MCP

10.3 Lazy vs. Eager Tool Loading

Connecting 10 MCP servers could easily register 300+ tools. If all 300 tool definitions were injected into the system prompt, context overhead would be immense.

Antigravity handles this by classifying MCP tools as **Lazy Loaded**:

- The agent is provided a meta-tool: `call_mcp_tool(server, tool, args)`.
- It reads the tool's JSON schema only when it decides to call it, keeping the primary prompt lean.

Part IV

Programmatic Agent Mastery & The Skill Stack (Expert)

Chapter 11

The Antigravity Python SDK

11.1 Headless Agent Orchestration

While the IDE and CLI cater to humans sitting at keyboards, the modern enterprise requires **headless agent pipelines**: automated PR reviewers, nightly security patchers, and autonomous triage bots.

Google provides the official Python SDK:

```
1 pip install google-antigravity
```

Listing 11.1: Installing the Python SDK



Figure 11.1: Multi-Agent Swarm Orchestration: Spawning autonomous subagents for automated testing, cloud pipelines, and browser automation.

11.2 Building an Asynchronous SRE Agent

Below is a complete, production-ready Python script using the SDK:

```
1 import asyncio
```

```

2 import sys
3 from google.antigravity import (
4     Agent,
5     LocalAgentConfig,
6     CapabilitiesConfig,
7     SecurityPolicy
8 )
9
10 async def run_diagnostics():
11     # Configure the programmatic agent
12     config = LocalAgentConfig(
13         workspace_dir="/var/www/ecommerce-core",
14         system_instructions=(
15             "You are an expert Site Reliability Engineer. "
16             "Inspect recent crash logs, locate the bug in source "
17             "code, "
18             "and create a pull request with unit test coverage."
19         ),
20         capabilities=CapabilitiesConfig(
21             allow_file_modifications=True,
22             allow_terminal_execution=True,
23             allow_internet_access=True
24         ),
25         security_policy=SecurityPolicy(
26             terminal_sandbox=True,
27             non_workspace_access="deny"
28         )
29     )
30     # Agent acts as an async context manager
31     async with Agent(config) as agent:
32         print("[+] Agent session established. Dispatching "
33             "mission...")
34
35         response = await agent.chat(
36             "Diagnose 500 errors reported in /var/log/app.err.log"
37         )
38
39         # Stream internal chain-of-thought reasoning
40         async for thought in response.thoughts:
41             print(f"[THOUGHT] {thought}")
42
43         # Intercept and log tool calls
44         async for tool_call in response.tool_calls:
45             print(f"[TOOL] Invoking {tool_call.name} with "
46                 f"{tool_call.args}")
47
48         # Stream final conversational tokens
49         print("\n--- Final Agent Report ---")
50         async for token in response:
51             sys.stdout.write(token)
52             sys.stdout.flush()
53         print()
54
55 if __name__ == "__main__":
56     asyncio.run(run_diagnostics())

```

Listing 11.2: Automated SRE Diagnostic Bot with google-antigravity

11.3 Deep Architectural Comparison: CLI vs. SDK

Understanding the boundary between `agy` and `google-antigravity`:

- **The CLI (`agy`):** Designed for interactive human sessions or one-shot command lines. The output is rendered via a terminal UI (Rich/TUI). Error handling stops and asks the user for guidance.
 - **The Python SDK:** Designed for programmatic integration into larger distributed systems. Yields strongly-typed async events (`ThoughtDelta`, `ToolCallEvent`, `StreamChunk`). Gives the host Python process complete programmatic control to intercept tool calls, inject synthetic results, or route tasks to worker queues.
-

Chapter 12

Swarms, Browser Agents & Daemons

12.1 Subagent Delegation (`invoke_subagent`)

When a task is too vast for a single context window, the primary agent can spawn specialized **Subagents**:

- **Context Isolation:** The subagent begins with a clean context, insulated from conversational noise in the main thread.
- **Specialized Personas:** Subagent A is spun up as a "Database Index Optimizer"; Subagent B is spun up as a "Frontend CSS Expert".
- **Return Value Consolidation:** When the subagent completes its task, it returns a concise summary back to the parent agent.

12.2 Autonomous Browser Automation

Testing web applications requires seeing what the user sees. Antigravity includes a dedicated browser automation subagent:

1. Launches a headless Chromium instance.
2. Navigates to local development URLs (e.g., `http://localhost:3000`).
3. Types into form inputs, clicks buttons, and asserts DOM changes.
4. **Automatic WebP Video Recording:** Automatically captures a recording of the entire browser session and embeds it in the walkthrough artifact so you can visually verify the UI test.

12.3 Background Daemons & The Scheduler

Rather than locking the terminal with `sleep 30`, Antigravity uses the `schedule` tool:

- **One-Shot Timers:** Fires a notification after N seconds, waking up the agent reactively.
- **Recurring Cron Jobs:** Configured with standard 5-field cron syntax (`*/30 * * * *`) to run automated health checks or polling jobs.

Engineering Lab Project: Project 4: AutoSRE Sentinel - Building a 24/7 Monitoring Agent

We create a standalone Python daemon using the Antigravity SDK:

1. The script connects to an AWS SQS queue receiving alerts from Datadog.
2. When an anomaly is detected, it spins up an **Agent** instance scoped to the offending microservice repo.
3. The agent inspects recent git commits, reproduces the failure in a container sandbox, runs headless browser verification with video recording, commits a hotfix to a branch, and opens a GitHub Pull Request with the WebP video attached.

Chapter 13

Enterprise Monorepos & CI/CD

13.1 Scaling Antigravity in Large Repositories

In monorepos containing millions of lines of code, global rules become unwieldy. Follow the **Domain-Partitioned Pattern**:

```
1 monorepo-root/
2 |-- .agents/
3 |   |-- rules/
4 |   |   \-- global-security.md      # Applies across all services
5 |   \-- hooks.json
6 |-- services/
7 |   |-- payments/
8 |   |   |-- .agents/
9 |   |   |   \-- rules/
10 |   |   |       \-- pci-dss.md      # Strict financial compliance rules
11 |   |   \-- AGENTS.md
12 |   \-- search/
13 |       |-- .agents/
14 |       |   \-- rules/
15 |       |       \-- elastic.md      # Indexing guidelines
16 |       \-- AGENTS.md
```

Listing 13.1: Monorepo directory layout for Antigravity

13.2 GitHub Actions CI/CD Integration

Run Antigravity as an automated pull-request review bot:

```
1 name: Antigravity Autonomous PR Review
2 on:
3   pull_request:
4     types: [opened, synchronize]
5
6 jobs:
7   agentic-review:
8     runs-on: ubuntu-latest
9     permissions:
10       contents: write
11       pull-requests: write
12     steps:
13       - name: Checkout Code
14         uses: actions/checkout@v4
```

```
15     with:
16         fetch-depth: 0
17
18     - name: Set up Python
19       uses: actions/setup-python@v5
20       with:
21         python-version: '3.11'
22
23     - name: Install Dependencies
24       run: |
25         pip install google-antigravity PyGithub
26
27     - name: Run Antigravity Review Bot
28       env:
29         GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
30         GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
31       run: |
32         python3 .agents/ci/review_pr.py
```

Listing 13.2: File: .github/workflows/antigravity_review.yml

Chapter 14

The AI Developer's Skill Stack: Thriving in the Agentic Era

14.1 The Evolution: From Code Typist to Systems Director

With the advent of autonomous coding agents, the economic value of memorizing programming syntax, library APIs, and boilerplate patterns has plummeted toward zero. Conversely, the value of **architectural decomposition, rigorous verification design, and systems-level orchestration** has grown exponentially.

The engineer of the future is not a typist: they are an **Executive Director of AI Engineering Swarms**.

14.2 The Six Core Pillars of AI Engineering

Core Architecture Concept: The Six Pillars of the Modern AI Engineer

To operate Antigravity at enterprise scale, engineers must develop mastery across six core dimensions:

1. **Test Harness Architecture (The Oracle Problem):** Constructing deterministic, fast, automated test suites that serve as the ground truth for agent verification.
2. **Attention & Context Curation:** Designing lean context boundaries, navigating AST symbol graphs, and eliminating prompt noise.
3. **Governance Stratification:** Intuitively knowing whether an instruction belongs in a conversational prompt, a hierarchical rule, a progressive skill, or a deterministic hook.
4. **Defensive Sandboxing & Blast Radius Isolation:** Hardening execution environments, establishing read/write jails, and restricting network capabilities.
5. **Agent Observability & Log Forensics:** Analyzing chain-of-thought streams, parsing agent transcripts, and diagnosing tool execution faults.
6. **Deconstructive Systems Decomposition:** Breaking complex product features into modular sub-tasks that can be leased to specialized subagents.

14.2.1 1. Solving the Oracle Problem

An AI agent can only generate code as good as the test suite that evaluates it. If your codebase has no tests, an agent operating under `/goal` will consider its job done as soon as the syntax compiles—regardless of whether business logic is broken. AI engineers spend 60% of their effort designing bulletproof test harnesses and property-based tests that act as objective truth oracles.

14.2.2 2. The Tri-Layer Governance Stratum

When establishing repository standards, use the following decision matrix:

- **Use Prompts:** For ephemeral, one-time instructions specific to the immediate task.
- **Use Rules (`.agents/rules/`):** For universal coding style, lint standards, and architectural conventions that must apply across multiple turns.
- **Use Skills (`.agents/skills/`):** For complex multi-step procedures (runbooks, deployments, migrations) loaded on-demand via progressive disclosure.
- **Use Hooks (`hooks.json`):** For non-negotiable security gates (e.g., blocking commits to main, scanning secrets) that must NEVER be bypassed by LLM hallucination.

Chapter 15

Interfacing Antigravity: WhatsApp, Telegram & Google Workspace

15.1 The Omnichannel Agent Architecture

In modern engineering teams, problems do not wait for developers to open an IDE. Critical deployment failures happen during off-hours, executive queries require instant telemetry summaries, and on-call engineers triage alerts on mobile devices.

To meet engineers wherever they are, Antigravity supports an **Omnichannel Gateway Architecture**:

[WhatsApp / Telegram] $\longleftrightarrow$ [Gateway Controller] $\longleftrightarrow$ [Antigravity Python SDK]
 $\longleftrightarrow$ [Google Sheets / Gmail / Codebases]

Core Architecture Concept: The Gateway Decoupling Pattern

Never connect conversational chat apps directly to raw bash toolkits without a mediating gateway. The Gateway Controller:

1. **Maps User Identities:** Resolves a Telegram `chat_id` or WhatsApp phone number to authorized repository permissions.
2. **Buffers Async Streaming:** Aggregates agent token deltas into conversational messages and handles Telegram/WhatsApp rate limits.
3. **Dispatches Artifacts:** Converts generated markdown reports, diff patches, or test videos into native mobile attachments.

15.2 WhatsApp Cloud API Integration Workflow

The Meta WhatsApp Business Cloud API operates over webhooks. Below is the complete production workflow for an autonomous Antigravity WhatsApp bot:

15.2.1 Production WhatsApp Gateway Server

```
1 import os
2 import httpx
3 from fastapi import FastAPI, Request, Query, Response
4 from google.antigravity import Agent, LocalAgentConfig,
   CapabilitiesConfig
```

Core Architecture Concept: Workflow: WhatsApp Inbound Triage

1. **Inbound Message:** User sends: *"Run smoke tests on staging and notify on-call"*.
2. **Webhook Receiver:** FastAPI validates the HMAC-SHA256 signature and parses the payload.
3. **Agent Execution:** The Antigravity Python SDK executes `pytest` inside the isolated sandbox.
4. **WhatsApp Response:** Formatted status and test summary are delivered back to the WhatsApp chat.

```

5
6 app = FastAPI()
7
8 WHATSAPP_TOKEN = os.environ["WHATSAPP_API_TOKEN"]
9 PHONE_NUMBER_ID = os.environ["WHATSAPP_PHONE_ID"]
10 VERIFY_TOKEN = os.environ["WHATSAPP_VERIFY_TOKEN"]
11
12 # 1. Verification Challenge (Meta Setup)
13 @app.get("/webhook")
14 async def verify_webhook(
15     mode: str = Query(..., alias="hub.mode"),
16     token: str = Query(..., alias="hub.verify_token"),
17     challenge: str = Query(..., alias="hub.challenge")
18 ):
19     if mode == "subscribe" and token == VERIFY_TOKEN:
20         return Response(content=challenge, media_type="text/plain")
21     return Response(status_code=403)
22
23 # 2. Inbound Message Handler
24 @app.post("/webhook")
25 async def handle_whatsapp_message(request: Request):
26     payload = await request.json()
27     try:
28         entry = payload["entry"][0]["changes"][0]["value"]
29         messages = entry.get("messages", [])
30         if not messages:
31             return {"status": "ignored"}
32
33         user_msg = messages[0]["text"]["body"]
34         sender_phone = messages[0]["from"]
35
36         # 3. Dispatch to Antigravity Agent
37         config = LocalAgentConfig(
38             workspace_dir="/var/www/ecommerce-api",
39             capabilities=CapabilitiesConfig(allow_terminal_execution=True)
40         )
41         async with Agent(config) as agent:
42             response = await agent.chat(user_msg)
43             full_text = ""
44             async for token in response:
45                 full_text += token
46
47         # 4. Reply via WhatsApp Graph API

```

```

48         url =
49             f"https://graph.facebook.com/v20.0/{PHONE_NUMBER_ID}/messages"
50         headers = {"Authorization": f"Bearer {WHATSAPP_TOKEN}"}
51         body = {
52             "messaging_product": "whatsapp",
53             "to": sender_phone,
54             "type": "text",
55             "text": {"body": full_text[:4096]}
56         }
57         async with httpx.AsyncClient() as client:
58             await client.post(url, json=body, headers=headers)
59     except Exception as e:
60         print(f"Error handling webhook: {e}")
61     return {"status": "ok"}

```

Listing 15.1: FastAPI WhatsApp Gateway for Antigravity

15.3 Telegram Bot Integration Workflow

Telegram provides superior support for long messages, markdown formatting, document uploads, and dynamic message editing.

15.3.1 Real-Time Thinking Stream over Telegram

Using the `response.thoughts` stream from the Antigravity Python SDK, we can update a Telegram message in real-time as the agent ponders:

```

1  import os
2  import asyncio
3  from telegram import Update
4  from telegram.ext import Application, CommandHandler,
5  MessageHandler, filters, ContextTypes
6  from google.antigravity import Agent, LocalAgentConfig,
7  CapabilitiesConfig
8
9  TELEGRAM_BOT_TOKEN = os.environ["TELEGRAM_BOT_TOKEN"]
10
11 async def handle_message(update: Update, context:
12     ContextTypes.DEFAULT_TYPE):
13     user_prompt = update.message.text
14     status_msg = await update.message.reply_text("Thinking...")
15
16     config = LocalAgentConfig(
17         workspace_dir="/home/dev/microservices/payment-hub",
18         capabilities=CapabilitiesConfig(allow_file_modifications=True,
19             allow_terminal_execution=True)
20     )
21
22     async with Agent(config) as agent:
23         response = await agent.chat(user_prompt)
24
25         # Stream thoughts to Telegram
26         thought_accumulator = ""
27         last_edit_time = asyncio.get_event_loop().time()
28
29         async for thought in response.thoughts:

```

```

26         thought_accumulator += thought
27         now = asyncio.get_event_loop().time()
28         # Throttle Telegram edits to prevent rate limits (max 1
           edit / 1.5s)
29         if now - last_edit_time > 1.5:
30             preview = thought_accumulator[-200:].strip()
31             await status_msg.edit_text(f"Working...\n\nThought:
           _{preview}_", parse_mode="Markdown")
32             last_edit_time = now
33
34         # Stream and accumulate final answer
35         final_answer = ""
36         async for token in response:
37             final_answer += token
38
39         # Deliver final message
40         await status_msg.edit_text(final_answer[:4000])
41
42 def main():
43     app = Application.builder().token(TELEGRAM_BOT_TOKEN).build()
44     app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND,
           handle_message))
45     print("[*] Antigravity Telegram Bot polling...")
46     app.run_polling()
47
48 if __name__ == "__main__":
49     main()

```

Listing 15.2: Telegram Bot with Real-Time Thinking Stream

15.4 Google Workspace Integration (Sheets & Gmail via MCP)

Antigravity seamlessly interacts with Google Sheets and Gmail through the **Model Context Protocol (MCP)** or direct Google API integrations.

15.4.1 1. Google Sheets as an Autonomous Audit Ledger

Configure the Google Sheets MCP server in `.agents/mcp_config.json`:

```

1  {
2      "mcpServers": {
3          "google_sheets": {
4              "command": "npx",
5              "args": ["-y", "@modelcontextprotocol/server-google-sheets"],
6              "env": {
7                  "GOOGLE_SERVICE_ACCOUNT_KEY_PATH":
8                      "/secrets/google_service_account.json"
9              }
10         },
11         "gmail": {
12             "command": "npx",
13             "args": ["-y", "@modelcontextprotocol/server-gmail"],
14             "env": {
15                 "GMAIL_CREDENTIALS_PATH": "/secrets/gmail_oauth.json"
16             }
17         }
18     }
19 }

```

18 }

Listing 15.3: Connecting Google Sheets MCP Server

15.4.2 2. Automated Bug Ledger & Executive Email Workflow

With Google Workspace MCP tools registered, an agent can execute multi-step cross-platform workflows:

Production Prompt Template: Multi-Platform Orchestration Prompt

Investigate failing payment webhook in `@file:logs/webhook_errors.log`.

1. Fix the bug in `src/webhooks/stripe.ts` and verify with `'npm test'`.
2. Append a new audit row to Google Sheet 'Production Incident Log' with timestamp, error summary, file changed, and author.
3. Draft an email in Gmail to 'engineering-leads@company.com' with subject '[INCIDENT RESOLVED] Stripe Webhook Fix' detailing the root cause and PR link.

Engineering Lab Project: Project 5: The Omnichannel SRE Concierge

An on-call engineer receives an alert on **WhatsApp** while commuting. They reply: *"@antigravity investigate high CPU on order service and post incident summary to Sheets"*.

1. The WhatsApp Gateway routes the prompt to the Antigravity agent.
2. The agent inspects container stats, identifies a runaway regex in `validator.py`, writes an optimized replacement, and runs the test suite.
3. The agent appends a structured log row to the shared team **Google Sheet**.
4. The agent drafts a post-mortem email in **Gmail** for the team.
5. The agent sends a **Telegram** message to the team channel with the git diff and sends a confirmation to the engineer on **WhatsApp** with the resolution status.

Appendix A: Slash Command Quick Reference

Table 15.1: Antigravity Slash Commands Reference

Command	Mode	Description
/help	All	Displays active tools, model configuration, and keybindings.
/goal	Autonomous	Locks the agent into self-verifying persistence mode until tests pass.
/grill-me	Planning	Launches an architectural interview to extract missing requirements.
/learn	Customization	Distills session insights into permanent workspace rules in <code>.agents/rules/</code> .
/schedule	Background	Configures delayed timers or recurring background cron jobs.
/clear	Context	Resets conversation history while retaining active workspace rules.
/exit	CLI	Gracefully terminates the session (<code>Ctrl+D Ctrl+D</code>).

Appendix B: Customization Directory Reference

Table 15.2: Antigravity Customization Directory Hierarchy

File / Path	Function & Precedence
<code>.agents/rules/*.md</code> <code>GEMINI.md</code> / <code>AGENTS.md</code>	Project-level rules. Highest precedence. Hierarchical directory rule files discovered during path walking.
<code>.agents/skills/<name>/SKILL.md</code>	Project skills. Loaded on-demand via progressive disclosure.
<code>.agents/hooks.json</code> <code>.agents/mcp_config.json</code>	Deterministic pre/post tool execution hooks. Project-level Model Context Protocol server configurations.
<code>~/.gemini/config/</code> <code>~/.gemini/antigravity-ide/</code>	Global user configuration root (machine-wide fallback). IDE application state, artifacts cache, and transcripts.

Appendix C: Keyboard Shortcuts Cheat Sheet

Table 15.3: Keybindings Across Antigravity Surfaces

Action	Antigravity IDE	Antigravity CLI (agy)
Accept Autocomplete	<Tab>	N/A
Trigger Inline Edit	Ctrl+I / Cmd+I	N/A
Open Chat Panel	Ctrl+Shift+L	Native TUI Canvas
Mention Context	@	@
Slash Commands	/	/
Cancel Execution	Escape	Ctrl+C
Exit Session	Close Window	Ctrl+D Ctrl+D

Appendix D: Production Prompt & Goal Cookbook

Production Prompt Template: Cookbook 1: Zero-Downtime Database Column Migration

/goal Objective: Add a new 'email_verified_at' TIMESTAMP column to the 'users' table without locking the production database.

Scope: Edits allowed only in db/migrations/ and src/models/user.ts.

Invariants:

1. Migration must be split into: (1) Add column as nullable, (2) Backfill script, (3) Model updates.
2. Provide both UP and DOWN SQL files.

Oracle: Execute 'npm run db:migrate:test' and 'npm test test/models/user.spec.ts'. All tests must pass.

Production Prompt Template: Cookbook 2: Performance Profiling & Optimization

Profile endpoint GET /api/v1/analytics/dashboard.

1. Run benchmarks with 'k6 run benchmarks/dashboard.js' and record P95 latency and memory usage.
2. Inspect @file:src/services/dashboard.service.ts for N+1 SQL queries or unindexed scans.
3. Optimize queries using DataLoader or Redis caching.
4. Re-run k6 benchmark. Optimization is successful only if P95 latency drops by > 40% and zero existing tests fail.

Production Prompt Template: Cookbook 3: Hardened API Endpoint Greenfield Scaffold

Scaffold a new endpoint: POST /api/v1/payments/refund.

1. **Strict Validation:** Use Zod schema to validate refund amount > 0, currency ISO 4217, and uuid v4 transaction_id.
2. **Idempotency:** Implement Redis idempotency key lock with 60s TTL.

3. **Architecture:** Split into handler (`src/api/refund.ts`), service (`src/services/refund.service.ts`), and audit log event.
4. **Tests:** Write unit tests in `test/refund.spec.ts` covering (a) success, (b) duplicate idempotency replay, (c) insufficient funds error.

Oracle: Run `'npm test test/refund.spec.ts'`. All tests must pass.